

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z)$$

$$= \nabla_{\phi} \mathbb{E}_{p_{\epsilon}} \log p_{\theta}(x|g(\epsilon)) \quad \text{By reparameterization}$$

$$= \nabla_{\phi} \frac{1}{\binom{M}{M}} \sum_{j=1}^M \log p_{\theta}(x|g(\epsilon_j))$$

for the same x_i , there can be multiple latent vec $\epsilon_1, \epsilon_2, \dots, \epsilon_M \sim p_{\epsilon}$

To compute: $\log p_{\theta}(x|g(\epsilon_j))$

parameterize $p_{\theta}(x|z)$ via some known distribution & use the decoder to output its parameters.

$$\text{example: } p_{\theta}(x|z) = \mathcal{N}(x; \hat{x}_{\theta}(z), \mathbf{I})$$

with the above.

$$\begin{aligned} \log p_{\theta}(x|z) &= \log \left\{ \frac{1}{(2\pi)^{d/2}} \exp \left(-\frac{1}{2} \|x - \hat{x}_{\theta}(z)\|_2^2 \right) \right\} \\ &\propto -\frac{1}{2} \|x - \hat{x}_{\theta}(z)\|_2^2 \end{aligned}$$

we need

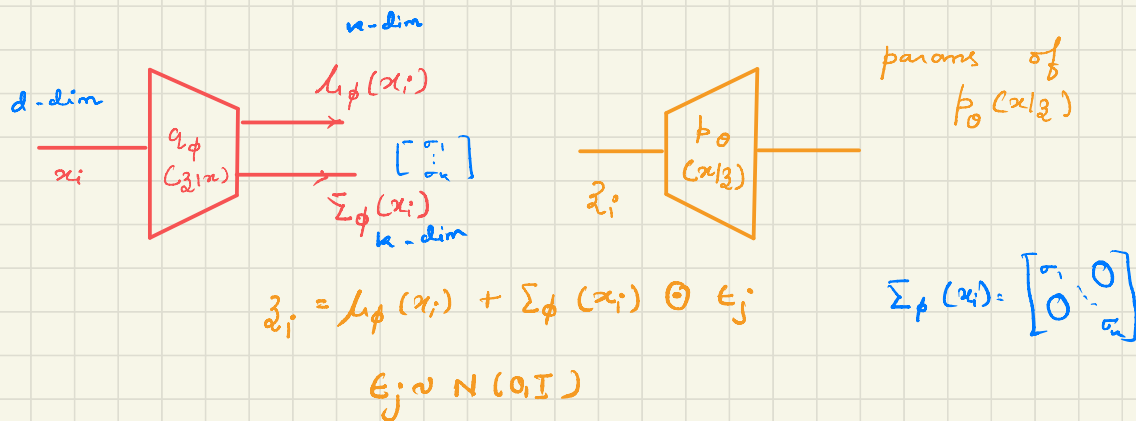
$$\nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z)$$

$$= \nabla_{\phi} \mathbb{E}_{p_{\phi}} \log p_{\theta}(x|g(\epsilon))$$

$$= \nabla_{\phi} \left\{ -\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_{\theta}(z_j)\|_2^2 \right\}$$

$$\text{where } z_j = \mu_{\phi}(x_i) + \Sigma_{\phi}(x_i) \odot \epsilon_j$$

$$\epsilon_1, \epsilon_2, \dots, \epsilon_M \sim N(0, I)$$



Forward pass

- 1) Given an $x_i \in D$
pass it through the Encoder, to obtain $\mu_{\phi}(x_i)$ & $\Sigma_{\phi}(x_i)$
- 2) Sample $\epsilon_1, \dots, \epsilon_M \sim N(0, I)$ outside the NNs.
- 3) Compute $z_1, \dots, z_M$ via reparameterization.
- 4) Pass all $z_1, \dots, z_M$ through the decoder

to compute $\hat{x}_\theta(z_j)$

$$\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_\theta(z_j)\|_2^2$$

Backward Pass:

1) Compute the grad

$$\nabla_\phi \frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_\theta(z_j)\|_2^2$$

2) Back prop through

Decoder : θ is kept constant

Reparameterization : Independent of ϕ .

Encoder :

Thus we obtain the $\nabla_\phi \mathbb{E}_{q_\phi(z|x)} \log p_\theta(x|z)$

To compute $\nabla_{\phi} D_{KL} [q_{\phi}(z|x) \parallel p_0(z)]$

$p_0(z)$: latent prior

Typically assumed to be $N(0, I)$

$$q_{\phi}(z|x) \sim N(z; \mu_{\phi}(x), \Sigma_{\phi}(x))$$

$$\therefore D_{KL} [q_{\phi}(z|x) \parallel p_0(z)]$$

$$= D_{KL} [N(\mu_{\phi}(x), \Sigma_{\phi}(x)) \parallel N(0, I)]$$

$$= -\frac{1}{2} \log |\Sigma_{\phi}(x)| - k + \text{tr}[\Sigma_{\phi}(x)] + \|\mu_{\phi}(x)\|_2^2$$

is a scalar

$$\therefore \nabla_{\phi} D_{KL} [q_{\phi}(z|x) \parallel p_0(z)]$$

$$= \nabla_{\phi} \left\{ -\frac{1}{2} \log |\Sigma_{\phi}(x)| - k + \text{tr}[\Sigma_{\phi}(x)] + \|\mu_{\phi}(x)\|_2^2 \right\}$$

$$= \nabla_{\phi} (KL)$$

$\therefore$ it will get added when updating parameters of Encoder

this term is independent of " θ "

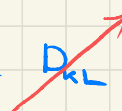
∴ Combining all together to update the Encoder parameter.

$$\phi^{t+1} \leftarrow \phi^t + \alpha \nabla_{\phi} J_{\theta}(\phi)$$

this is one training step through Encoder.

Training the Decoder

$$\nabla_{\theta} J_{\theta}(\phi) = \nabla_{\theta} \left[\mathbb{E}_{\phi(z|x)} \log p_{\theta}(x|z) \right] + \nabla_{\theta} D_{KL}$$

hapt of θ 

→ Forward pass is same as Encoder

→ Reverse path is stopped post decoder, where the θ are not kept cont unlike encoder updation.

$$\theta^{t+1} \leftarrow \theta^t + \alpha \nabla_{\theta} J_{\theta}(\phi)$$

Inference with Trained VAEs:

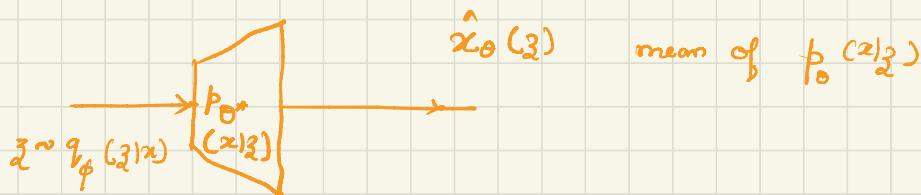
1) Data Generation / Sampling

Consider the following sampling procedure

$$p_0(x) = \int_{\mathbb{Z}} p_0(x|z) dz.$$

* Sample $z_{\text{new}} \sim p_z \sim N(0, I)$

* Sample $x_{\text{new}} \sim p_0(x|z)$ using decoder of VAE



The KL term in ELBO will ensure that

$$q_\phi(z|x) = p(z) = N(0, I) \quad \forall x$$

If VAE is well trained $q_\phi(z|x) = N(0, I) \quad \forall x$

$\Rightarrow$ Sampling from $N(0, I) \Rightarrow$ Sampling from $q_\phi(z|x)$

$\hat{x}_\theta(z_{\text{new}})$ is the output of decoder from $z_{\text{new}} \sim N(0, I)$

+ either use $\hat{x}_\theta(z_{\text{new}})$ as the novel generated datapoint

$$\hat{x}_\theta(z_{\text{new}}) \in \mathbb{R}^d$$

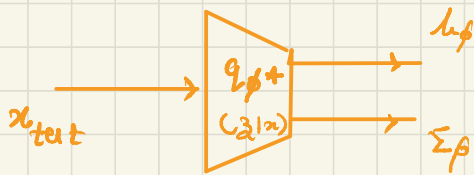
$$\textcircled{ii} \quad x_{\text{new}} \sim N(\hat{x}_\theta(z_{\text{new}}), I)$$

$$p_\theta(x|z) = N(\hat{x}_\theta(z), I)$$

Sample z obtain the new point which is the now generated data point.

2) VAEs for latent posterior inference $\textcircled{i}$ Embedding $\textcircled{ii}$ feature extraction.

done via encoder model, post training.

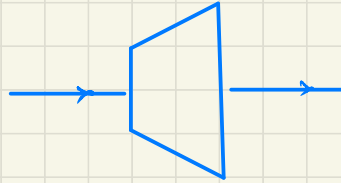
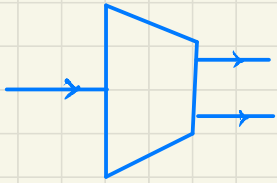


The embedding $\textcircled{i}$ the feature vector for x_{test} is given by

$$\text{(i)} \quad z_{\text{test}} = \mu_\phi(x_{\text{test}})$$

$$\text{(ii)} \quad z_{\text{test}} \sim N(\mu_\phi(x_{\text{test}}), \Sigma_\phi(x_{\text{test}}))$$

Posterior - Collapse in Naive VAE



* In VAE, the second term in ELBO

$D_{KL}(q_\phi(z|x) \parallel p(z))$ is minimized.

* $\forall x$ $q_\phi(z|x)$ is forcefully made to be the same as

$$p(z) \sim N(0, I)$$

This collapse to a specific distribution.

$\therefore$ The decoder will see it difficult to differentiate b/w
2 i/p samples x_i & x_j

$$\therefore q_\phi(z|x_i) = q_\phi(z|x_j) = N(0, I)$$

* forcing things to normal dist

& both z_i & z_j are from same dist

$$N(0, I)$$

Solution for Posterior Collapse in VAE

$$J_0(q_\phi) = \underbrace{\|x - \hat{x}_\theta(z)\|_2^2}_{\text{Recon term}} + \underbrace{D_{KL}(q_\phi(z|x) \parallel p(z))}_{\text{Regularization term on } z}$$

Restricting the behaviour

* VAE can be viewed as an Auto-Encoder with a regularized latent space

(Regularized Auto Encoder)

* Typical regularized cost in ML looks like

$$\underbrace{L_0(\theta)}_{\text{Cost func}} + \underbrace{\lambda \Omega(\theta)}_{\text{regularizing func}}$$

* In a VAE, viewed as regularized AE, the regularization const is absent.

* VAE can be made to trade-off b/w const & regularization by adding a regularization const to ELBO

$$J_0(q_\phi) = \|x - \hat{x}_\theta(z)\|_2^2 + \beta D_{KL}(q_\phi(z|x) \parallel p(z))$$

β -VAE. $0 \leq \beta \leq 1$, β -hyper parameter.

higher $\beta \rightarrow$ lead to posterior collapse, better generation

lower $\beta \rightarrow$ reconstruction are better

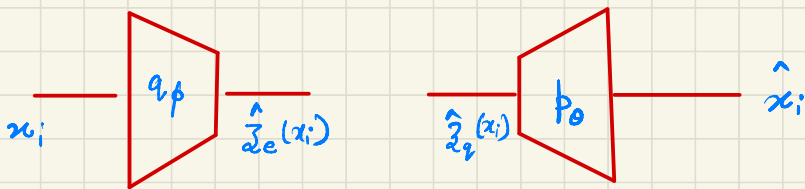
generation will suffer if $q(z|x) \neq p(z)$

Vector Quantized VAE (Vq-VAE).

* In Vq-VAE, the latent space is discrete and vector

vector quantized using a learnable dictionary.

* fundamental structure is discrete tokens.



$L = \{z_1, z_2, \dots, z_m\}$ latent dictionary.

$z_i \in \mathbb{R}^k$, $\hat{z}_e(x_i) \in \mathbb{R}^{k \times p}$

* Suppose there are M latent vectors of k -dim

$$z_q(x_i) = z_{j^*}, \quad j^* = \underset{j=1, \dots, M}{\operatorname{argmin}} \|z_e(x_i) - z_j\|_2^2$$

Vector quantization.

* In practice, the latent space is designed to be a tensor of latent dict

$$\hat{z}_e(x_i) = \begin{bmatrix} | & | & | & | \end{bmatrix}_{k \times p}$$

each row is a vector from L

each column of $\hat{z}_e(x_i)$ is vector quantized.

Training

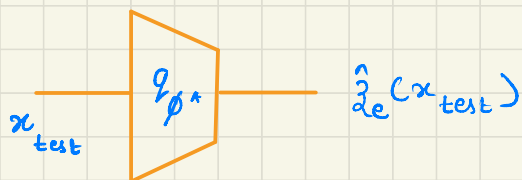
$$J_\theta(q) = \|x - \hat{x}_\theta(\hat{z}_e)\|_2^2 + \|z_e(x_i) - z_d(x)\|_2^2$$

In addition to the encoder & decoder parameters, the latent dictionary is also learnt.

$$z_j^{t+1} \leftarrow z_j^t - \alpha \nabla_{z_j} (J_\theta(q))$$

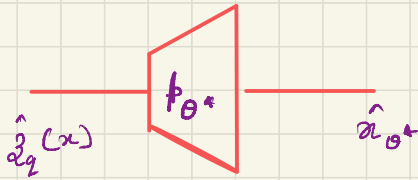
Inference :

a) Embedding extraction / Posterior Inference



$$\hat{z}_e(\text{test}) = \underset{j=1, \dots, m}{\text{argmin}} \| \hat{z}_e(x_{\text{test}}) - z_j \|_2^2$$

b) Sampling in Vq-VAE



To be able to sample from the decoder of Vq-VAE we need to sample from the distribution of $\hat{z}_z()$

* To learn to sample from the dist of $\hat{z}_z()$, a generative model is fit to samples of z_z

Eg : GMM on the samples from the o/p of encoder corresponding to all i/p datapoints.

Sample from the learned dist of z_z & use it as the input to the decoder